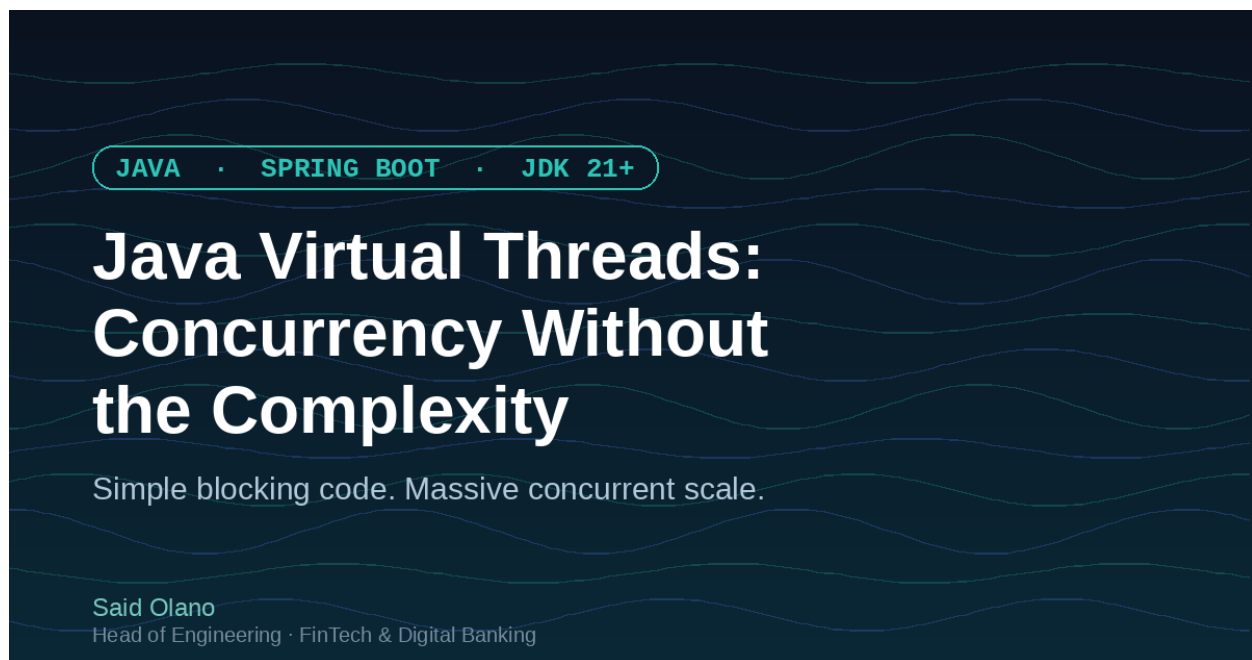




Java Virtual Threads: Concorrenza Senza la Complessità

Di Said Olano - Head of Engineering, FinTech & Digital Banking

Se costruisci servizi Java da un po', conosci bene questo compromesso: thread-per-request è semplice ma non scala oltre poche migliaia di connessioni concorrenti, mentre la programmazione reattiva (Project Reactor, Mono/Flux, catene di callback) scala magnificamente ma trasforma il tuo codice in qualcosa che solo metà del team riesce a debuggare.

JDK 21 ci offre una terza opzione: i Virtual Threads, introdotti tramite Project Loom. Ti permettono di continuare a scrivere Java semplice e bloccante, mentre la JVM gestisce la ginnastica di scheduling che prima richiedeva un framework reattivo.

Il problema della concorrenza tradizionale

Un thread di piattaforma (del sistema operativo) è costoso. Ognuno riserva circa 1MB di spazio di stack e coinvolge il sistema operativo a ogni cambio di contesto. Se avvii un thread per ogni richiesta in arrivo, un server collassa da qualche parte tra poche migliaia e diecimila connessioni concorrenti — molto prima che la CPU stessa diventi il collo di bottiglia.

La soluzione tradizionale è stata il codice asincrono e non bloccante: stream reattivi, callback, catene di `CompletableFuture`. Funzionano, ma cambiano il modo in cui scrivi e leggi il codice, come esegui il debug degli stack trace e come ragioni sui fallimenti.

Ecco i virtual thread

Un virtual thread è un thread leggero gestito dalla JVM anziché dal sistema operativo. Puoi crearne milioni, perché non vengono mappati 1:1 sui thread del sistema operativo — la JVM multiplexa molti virtual thread su un piccolo pool di thread "carrier", mettendo in pausa un virtual thread ogni volta che si blocca su I/O e liberando il thread carrier per servire qualcun altro.

Il risultato: scrivi normale codice bloccante —

```
public String fetchUser(String id) {  
    var response = httpClient.send(request, BodyHandlers.ofString());  
    return response.body();  
}
```

— e il runtime ti offre le caratteristiche di throughput del codice asincrono, senza il carico mentale.

Attivarlo in Spring Boot

A partire da Spring Boot 3.2, attivare i virtual thread per il tuo web server e i tuoi task executor è una modifica di una sola riga:

```
spring.threads.virtual.enabled=true
```

Tutto qui. I tuoi endpoint `@RestController`, le chiamate JDBC (con un driver JDBC 4.3+) e le chiamate `RestClient`/`RestTemplate` ora girano su virtual thread per impostazione predefinita. Nessuna riscrittura del codice, nessun tipo reattivo che trapela nel tuo livello di servizio.

Dove brillano

I virtual thread sono pensati per carichi di lavoro I/O-bound: API REST che chiamano servizi a valle, query al database, accesso ai file, qualsiasi cosa che passi la maggior parte del tempo in attesa piuttosto che a calcolare. Questo è esattamente il profilo della maggior parte dei microservizi Spring Boot.

Dove non aiutano

Non sono una soluzione magica per il lavoro CPU-bound. Eseguire un virtual thread non rende più veloce una moltiplicazione di matrici o una serializzazione JSON — hai comunque lo stesso numero di core CPU. Se il tuo servizio è computazionalmente intensivo, i virtual thread non cambieranno molto le cose.

L'insidia: il pinning dei thread

I virtual thread possono rimanere "pinnati" (pinned) al loro thread carrier in un paio di situazioni — soprattutto all'interno di un blocco o metodo `synchronized` che si blocca. Mentre è pinnato, il thread carrier non può essere riutilizzato, il che vanifica lo scopo. La soluzione è solitamente semplice: sostituire `synchronized` con `java.util.concurrent.locks.ReentrantLock` nei percorsi critici che girano su virtual thread. (JDK 24 rimuove gran parte di questa limitazione, ma se sei sulla LTS 21, tienilo a mente.)

Per concludere

I virtual thread sono una delle rare funzionalita della JVM che offrono un reale vantaggio di scalabilita al costo di un flag di configurazione e un po' di attenzione ai casi limite. Se esegui Spring Boot su JDK 21+, vale la pena testarlo in un ambiente di staging prima del tuo prossimo load test.

Hai gia adottato i virtual thread in un servizio Spring Boot in produzione? Mi piacerebbe sapere cosa hai incontrato — lascia un commento qui sotto.